

Global Infrastructure & Networking

A practical continuation of the AWS foundations reference

Table of Contents

RECAP	Global infrastructure Regions, Availability Zones and AWS global reach from Part I.
01	Networking concepts VPCs, subnets, addressing, routing and traffic control.
02	Practice Lab Build the VPC through traffic

Global Infrastructure

In Part 1, we learned how AWS can store files and run applications.

But running an application is only the beginning.

Imagine we're building an online store. Customers need to reach our web servers from the internet, while the web server needs to communicate with a database containing customer and order information.

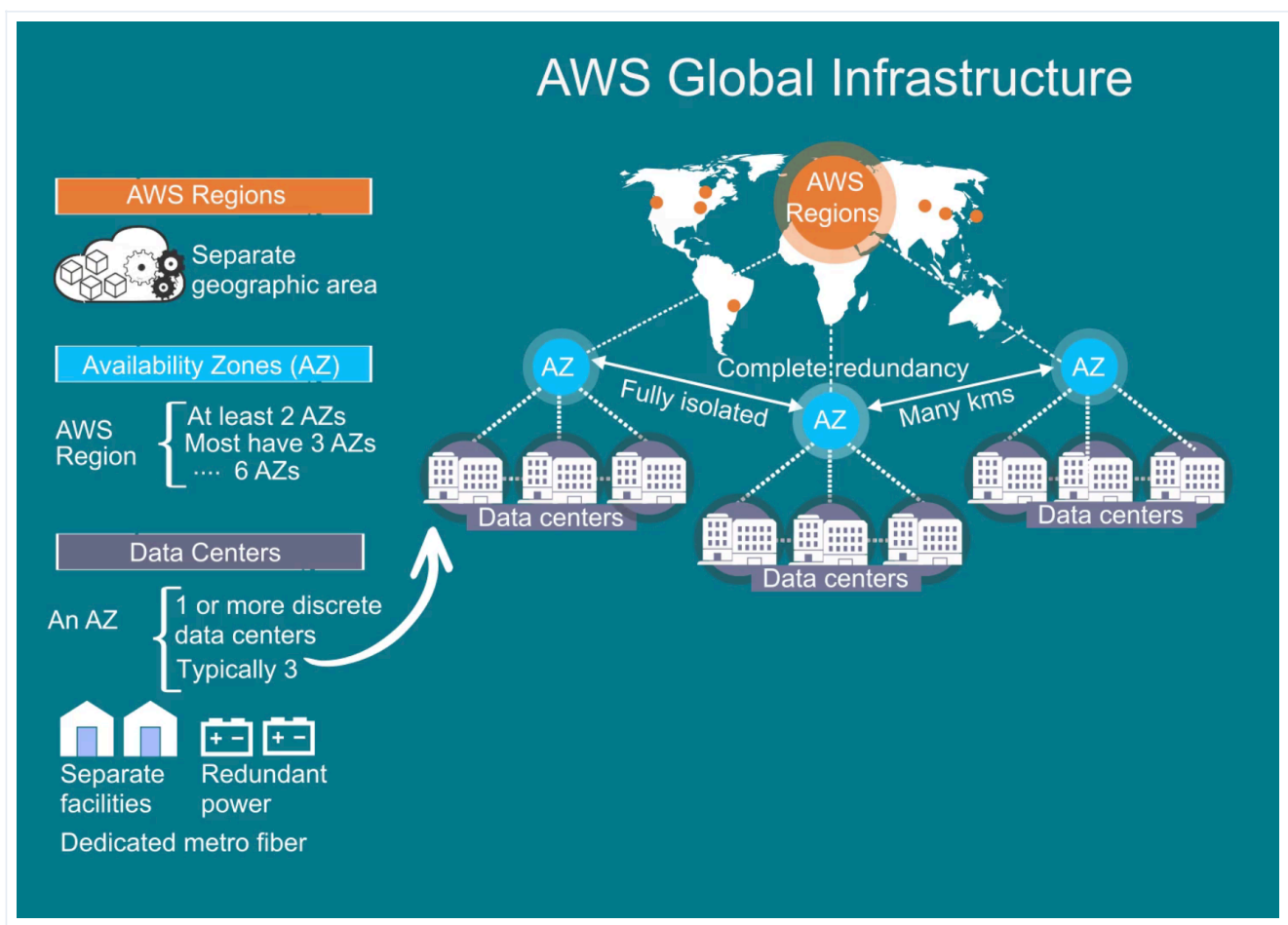
We don't want that database exposed directly to the internet, and we don't want the entire application to disappear because one location fails. But we also want to deliver resources across the globe efficiently.

So before adding more services, we need to design where our resources run and how traffic moves between them.

This is where AWS infrastructure and networking come together.

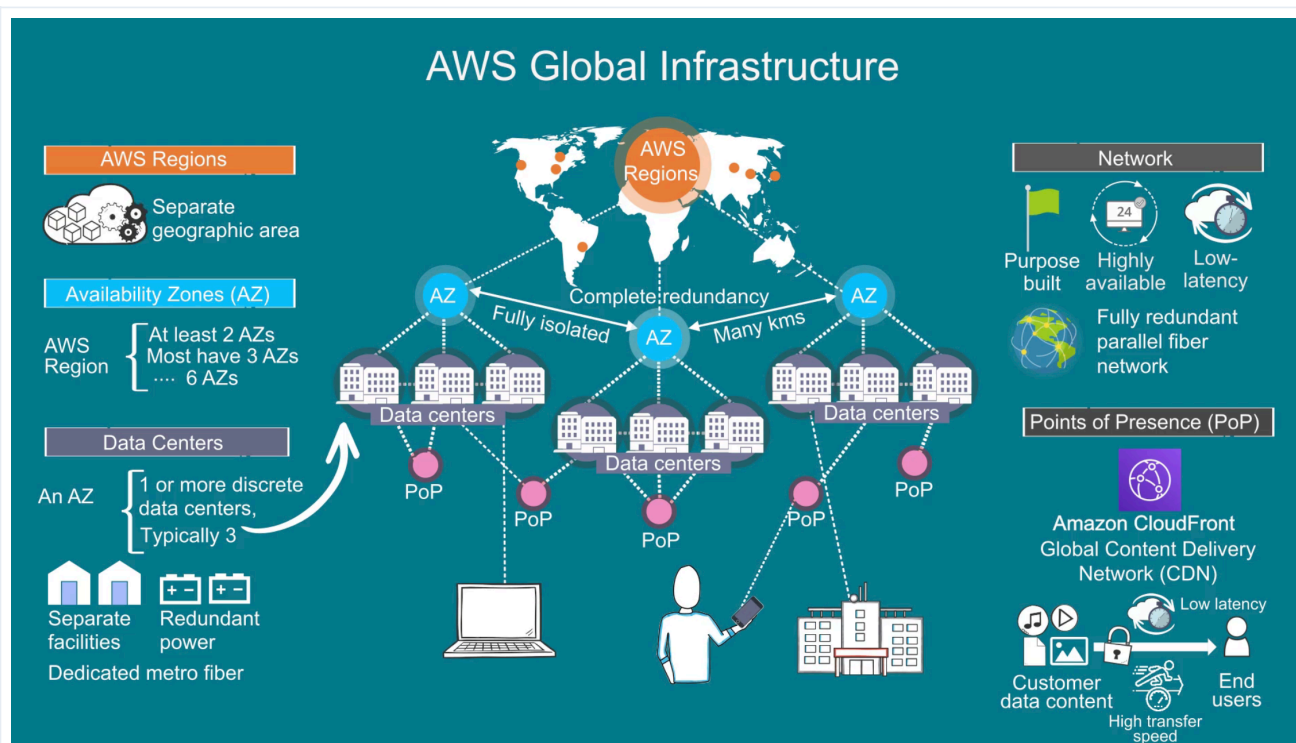
AWS is not one giant data center.

It is a worldwide infrastructure made of separate geographic Regions, with isolated Availability Zones inside them and global networking that connects services and customers.



Points of Presence and global delivery

Regions and Availability Zones are where workloads run, but customers do not always need to connect directly to a Region. AWS also operates Points of Presence (PoPs) around the world. Services such as CloudFront use this edge infrastructure to deliver content closer to end users.



EDGE DELIVERY

PoPs bring content closer to users

- A Point of Presence is an AWS edge location used by services such as CloudFront.
- CloudFront can cache copies of content at edge locations, reducing the distance many user requests need to travel.
- On a cache miss, CloudFront retrieves the content from the configured origin, such as S3 or an application, and can cache the response for later requests.
- AWS connects its infrastructure through a highly available, redundant global network, helping traffic move between edge locations, Regions and AWS services.

Mental model:

Region/AZ = where your workload runs.

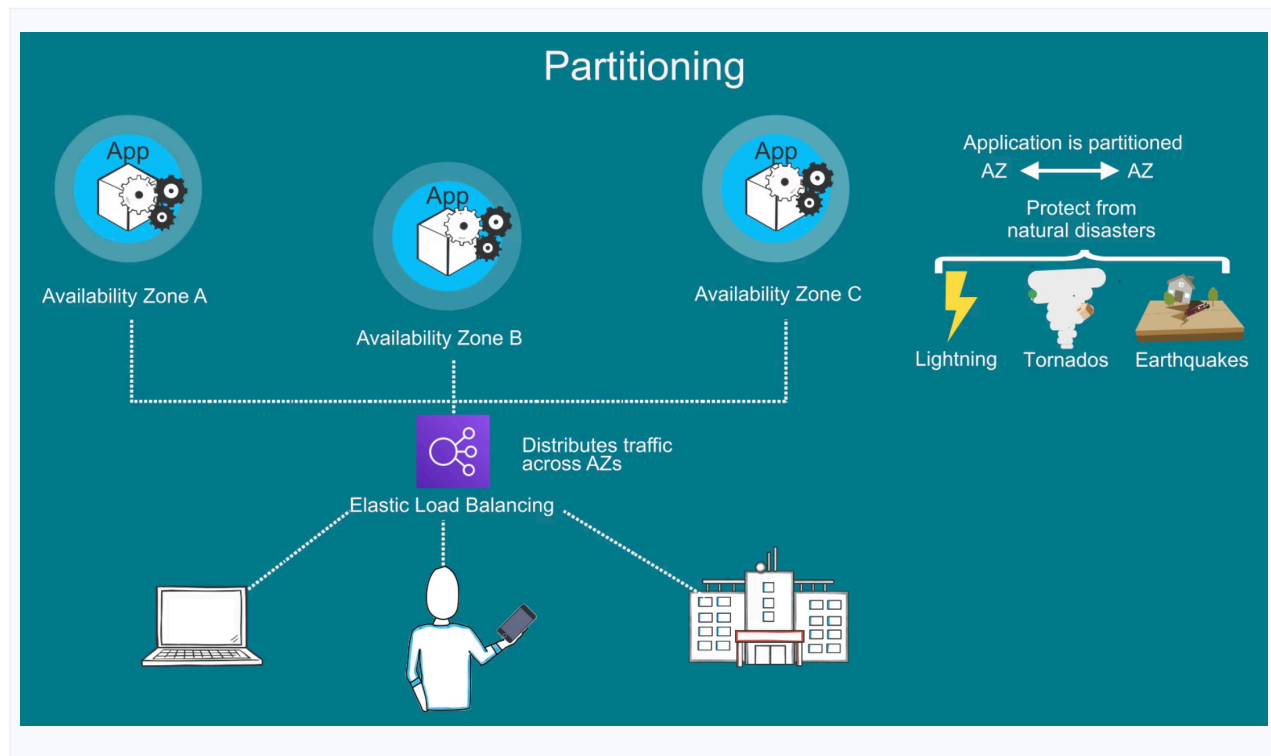
PoP/edge = where services such as CloudFront can meet users closer to them.

Why multiple Availability Zones matter

Imagine your entire application runs in **one Availability Zone**. You might have several EC2 instances, but they still depend on the same location. If a catastrophe hits that Availability Zone and it experiences an outage, all of those instances could become unavailable at the same time.

To reduce this risk, we can **partition the application across multiple Availability Zones**.

In simple terms, this means placing copies of the application in separate AZs rather than keeping everything in one.



NOTE

Elastic Load Balancing will be included in the final module. Keep the focus on partitioning and delivery.

An Availability Zone itself does not have a price.

AWS does not charge you simply because your VPC or application uses multiple AZs.

What costs money are the resources you place in those AZs and, in some cases, the data transferred between them.

For example, imagine our application originally has one EC2 instance in one AZ. If we improve availability by running another EC2 instance in a second AZ, we are now paying for two EC2 instances. The additional cost comes from the

What the global infrastructure gives us

AWS can place infrastructure closer to customers, spread workloads across isolated failure domains and adjust capacity as needs change. The result is a platform designed around reach, reliability, performance and flexibility without owning the physical infrastructure yourself.

t3.micro	m4.large
0.0104 USD per Hour	0.1000 USD per Hour
0.0196 USD per Hour	0.1920 USD per Hour
2 (1 core)	2 (1 core)
1024	8192
–	–
ebs	ebs
Up to 5 Gigabit	Moderate

Flexibility at the compute level

In Part I we stopped an EC2 instance, reviewed a different compatible instance type and could replace its compute capacity with a larger option.

That is one small example of the wider cloud idea: infrastructure is not fixed hardware you bought years ago. It can be adjusted as the workload changes.

NOTE

AWS expanding its footprint does not mean your resources automatically move into new Regions or AZs. You choose where your workload is deployed. The expanding footprint gives you more placement options over time.

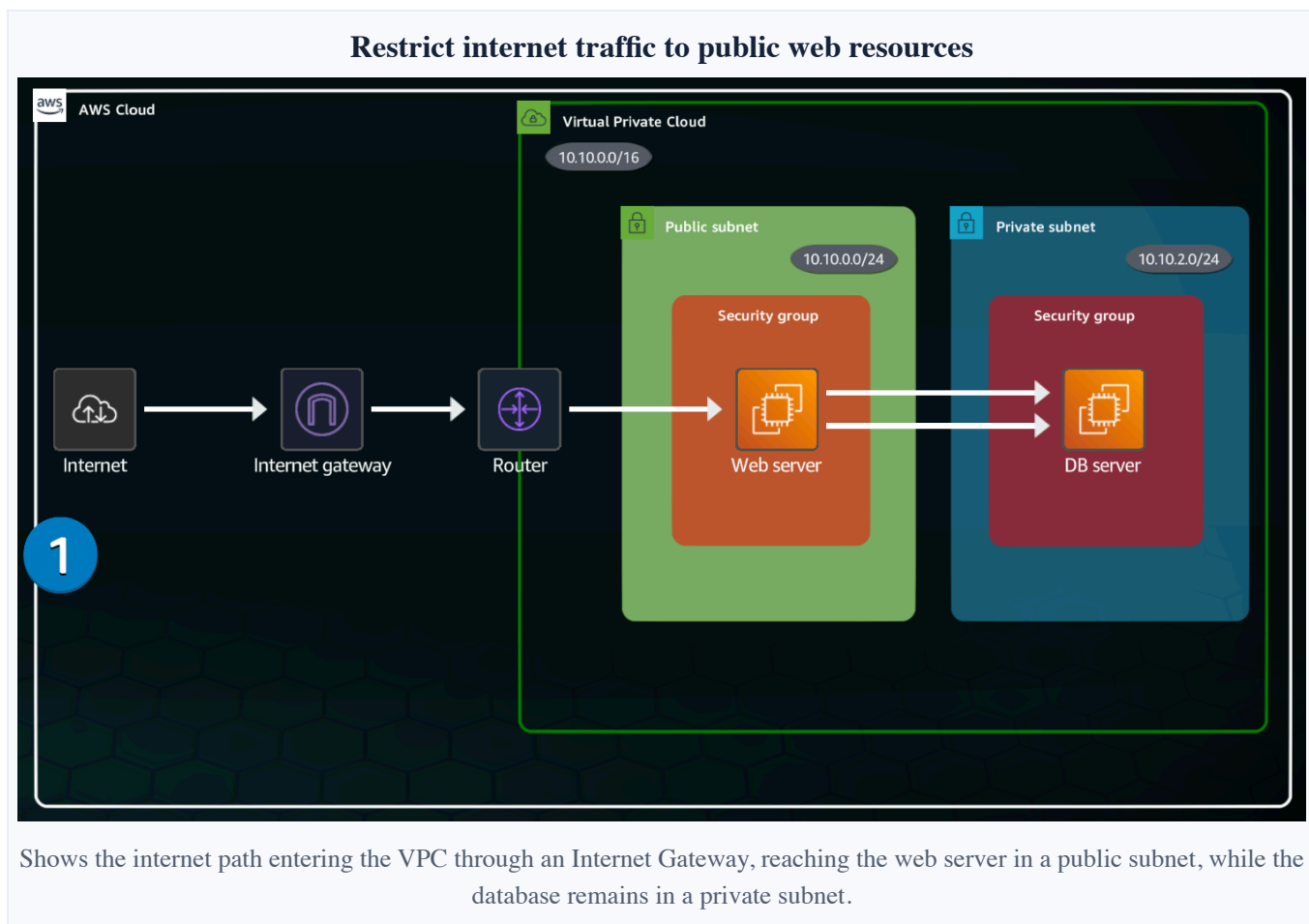
01 Networking Concepts

Now that we know where AWS resources can run, the next question is how they communicate. We start with the VPC, then narrow the network into subnets before looking at routes, addresses and traffic rules.

From internet traffic to a VPC

The first networking idea is separation.

A customer-facing web server may need a path from the internet, while a database usually should not. AWS networking lets us decide which resources are exposed and which remain private.



Separation

The architecture keeps the web server and database inside the same VPC, but gives them different network roles. The web tier can accept the traffic it needs from outside; the database can be reached by the application without being directly exposed to the internet.

What is a VPC?

Imagine you want to run a web server and a database in AWS.

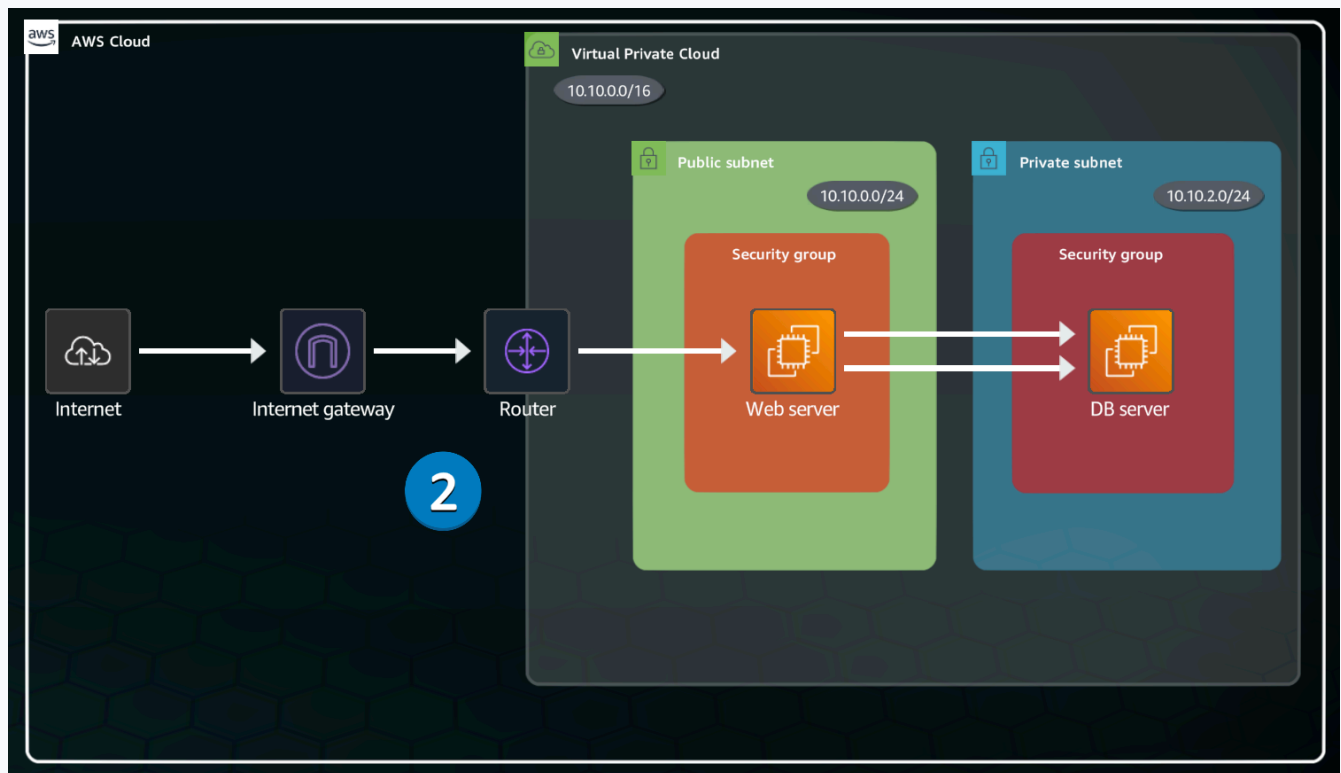
How do you decide which resources can reach the internet, which can communicate with each other, and which should stay private?

You could manage each resource individually, but that quickly becomes difficult as the application grows. A VPC gives you one network environment in which to organise and control all of this.

A **Virtual Private Cloud (VPC)** is a logically isolated virtual network in one AWS Region. It gives your AWS resources an IP address range and provides the network structure they use to communicate. You then divide that range into subnets, define routes, attach gateways when needed, and control traffic with security rules.

AWS provides a default VPC in each Region so resources can be launched quickly, but you can also create your own VPC when you want control over the address range and network design.

The application network inside a VPC



Public and private subnets divide the network, while **Security Groups** control allowed traffic.

NOTE

All you need to know at this point is that VPC is the container for both our Web Server and Database Server.

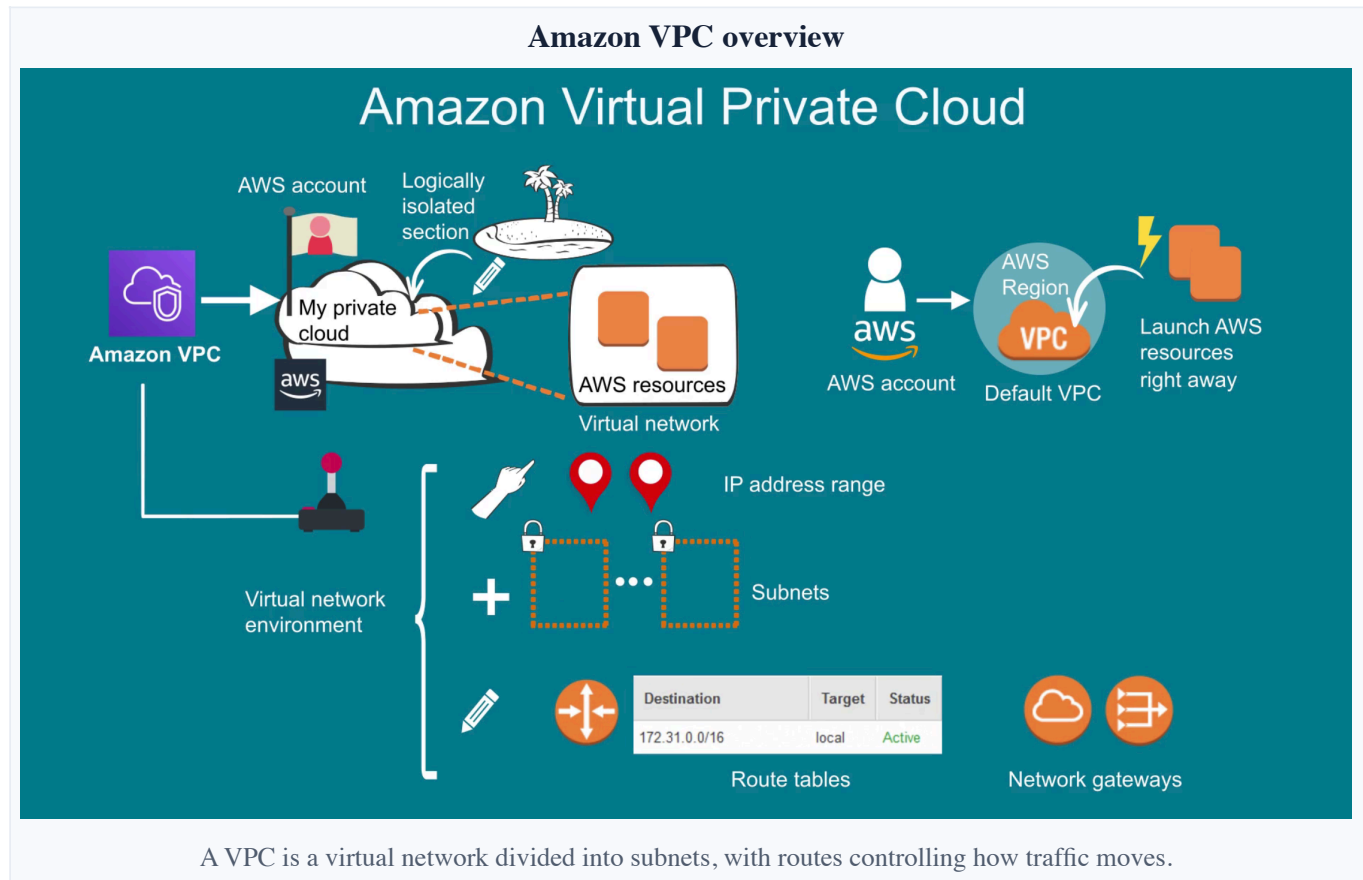
VPCs, Subnets and Availability Zones

We now have two ideas:

Availability Zones give us separate locations for our resources.

The VPC gives those resources a network in which to communicate.

Subnets connect these ideas.



A VPC spans the Region, but each subnet belongs to one **Availability Zone**. Therefore, if we want our application to run across multiple AZs, we create a subnet in each one.

So a region contains a VPC with multiple Availability Zones

Availability Zone A: Subnet A: 10.10.1.0/24

Availability Zone B: Subnet B: 10.10.2.0/24

A VPC spans Availability Zones while a subnet does not.

VPC & Subnet

We know that a VPC can contain several subnets.

But if they all belong to the same VPC, how does each subnet get its own part of the network?

This is where IP ranges come in.

The VPC starts with one larger range of IP addresses. When you create a subnet, you give it a smaller range taken from the VPC's range. For example:

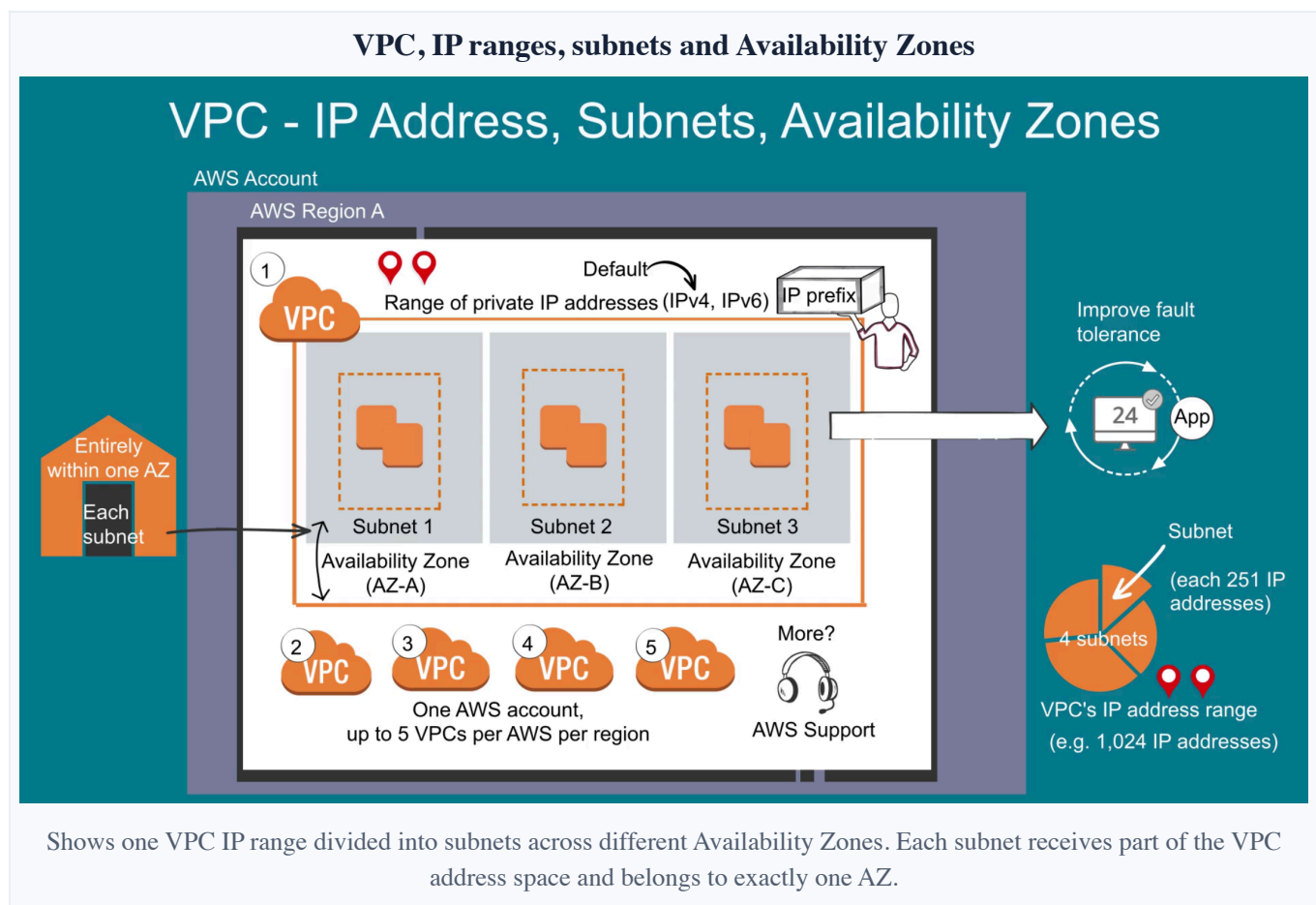
VPC: 10.0.0.0/16

Subnet A - 10.0.1.0/24 (AZ-A)

Subnet B - 10.0.2.0/24 (AZ-B)

So there are really two things happening at once: Availability Zone is where the subnet exists. IP Range is which addresses belong to the subnet.

A subnet defines where your resources run and which IP addresses they can use. This is important because a subnet determines where your resources can run and which network they belong to, helping you organise resources across Availability Zones and control how traffic moves between them.



Public and private traffic

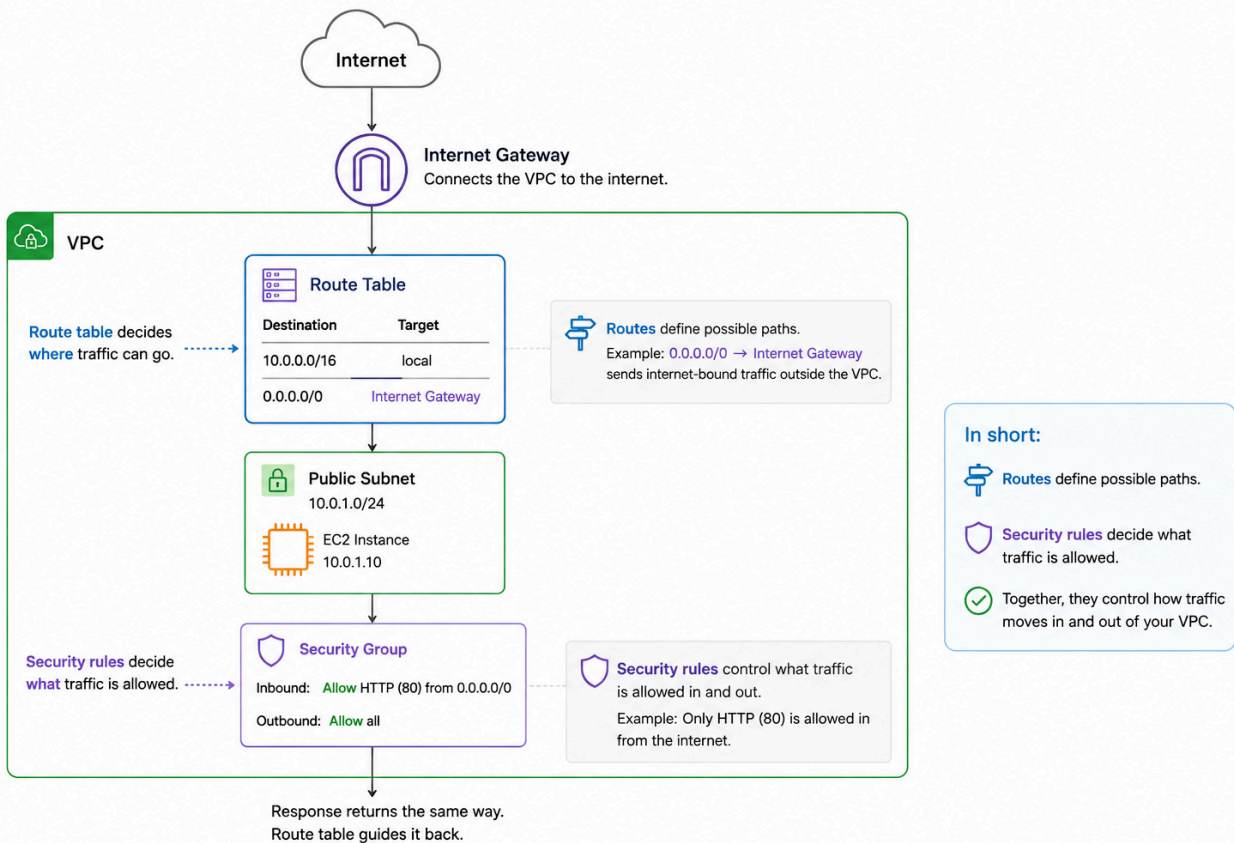
Not every resource in our VPC should communicate with the internet the same way. Our web s servers needs to receive requests from customers, while our database should remain private. This creates two different networking needs: public traffic and private traffic.

TRAFFIC PATH

Routes and gateways

A route table tells traffic where it can go. An Internet Gateway connects a VPC to the internet when routing and security allow it.

Routes define possible paths. Security rules decide what traffic is allowed.



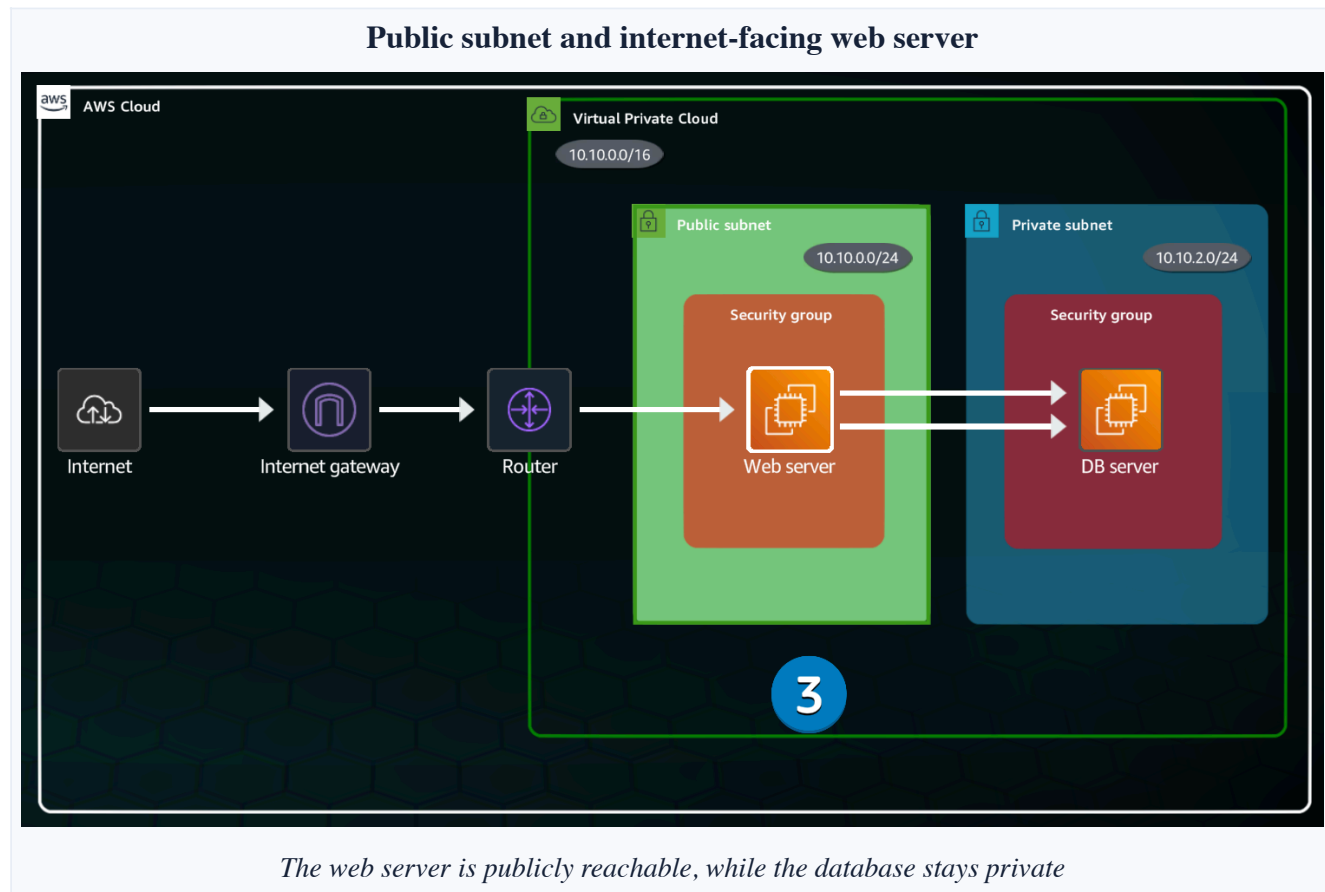
NOTE

Public and Private are determined by network paths, not simply by the resource itself. Routes determine where traffic can go, while Security Groups determine whether that traffic is allowed.

Subnets, CIDR blocks and routing

We now have the network structure we wanted: a **web server that customers can reach** and a **database that remains private**.

But AWS doesn't make a subnet public or private simply because we call it that. A step remaining is defining where traffic is actually able to go.



This brings together three concepts:

Subnet

A smaller network inside the VPC, located in one Availability Zone.

CIDR block

Defines the IP address range, such as `10.10.0.0/16`

Route table

Decides where network traffic should go next

For the application, the important difference is **routing**. The web server's subnet can have a route to the internet, while the database subnet can remain without a direct internet route.

What to remember

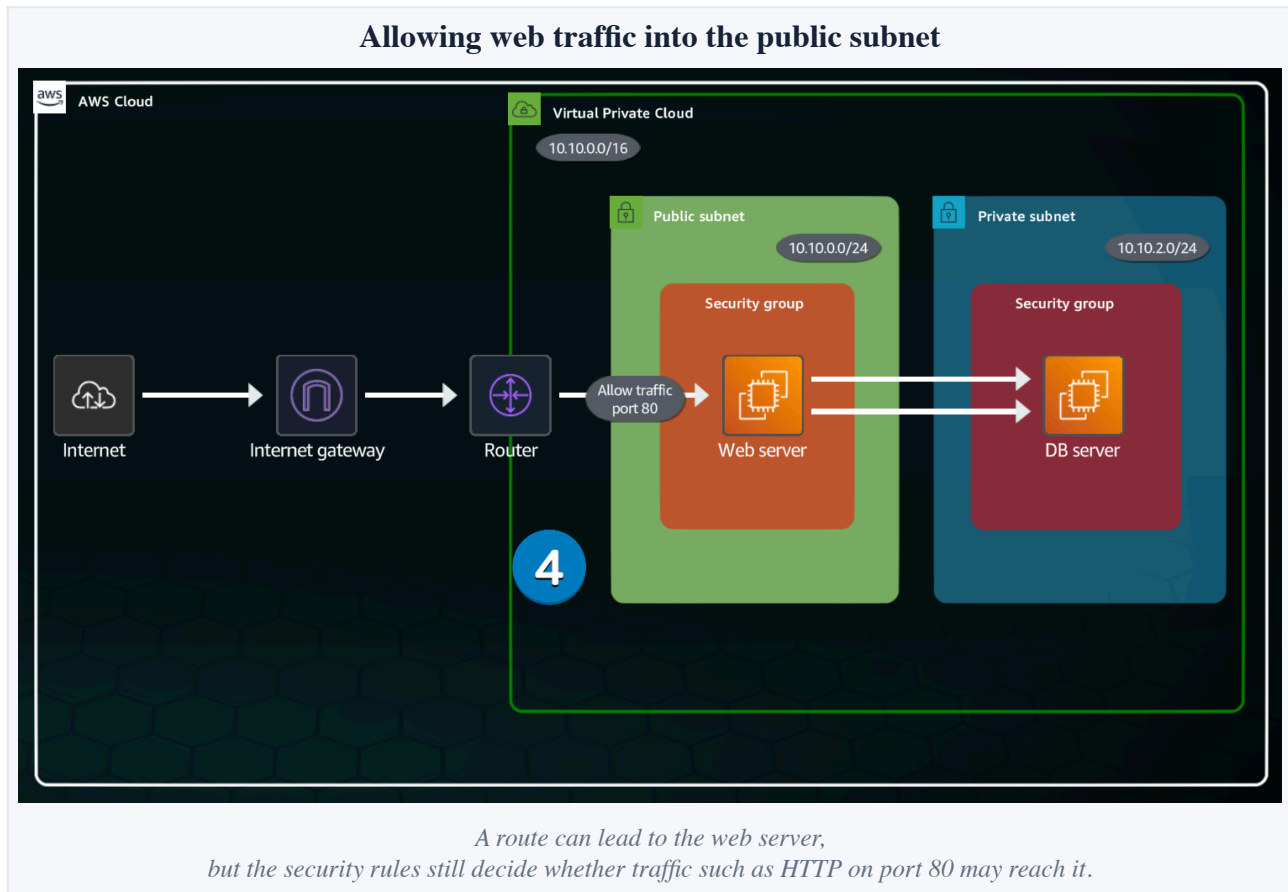
- A VPC is the network boundary for your resources in one Region.
- A subnet belongs to one Availability Zone and uses part of the VPC IP range.
- Routes define possible traffic paths; Security Groups decide what traffic is allowed.
- Private IPs work inside the VPC. Public addressing and DNS make resources reachable in other ways.

Security Groups and Network ACLs

So far, we've decided **where traffic can go** using routes and gateways.

But having a path doesn't mean traffic should automatically be allowed. AWS provides security controls at two levels: **Security Group & NACL**.

For most applications, **Security Groups** are the **primary way we control access to resources**.



Protecting the resource

A **Security Group** acts as a virtual firewall around resources such as EC2 instances. It controls two directions: **Inbound & Outbound**.

Inbound > Traffic trying to reach your resource.

Outbound > Traffic your resource is sending elsewhere

Direction	Example rule	Meaning
Inbound	HTTP · TCP · 80 · 0.0.0.0/0	Allow IPv4 web traffic from anywhere
Inbound	SSH · TCP · 22 · trusted IP	Allow SSH from a specific source
Outbound	All traffic	Allow the instance to initiate outbound traffic

For a web server, customers visiting the website are **inbound**, while calling an external API is **outbound**.

Allowed doesn't mean reachable

A Security Group only decides whether traffic is **allowed**. It does not create the network path.

Security Group: “You are allowed”

Route Table: “Here’s where you can go”

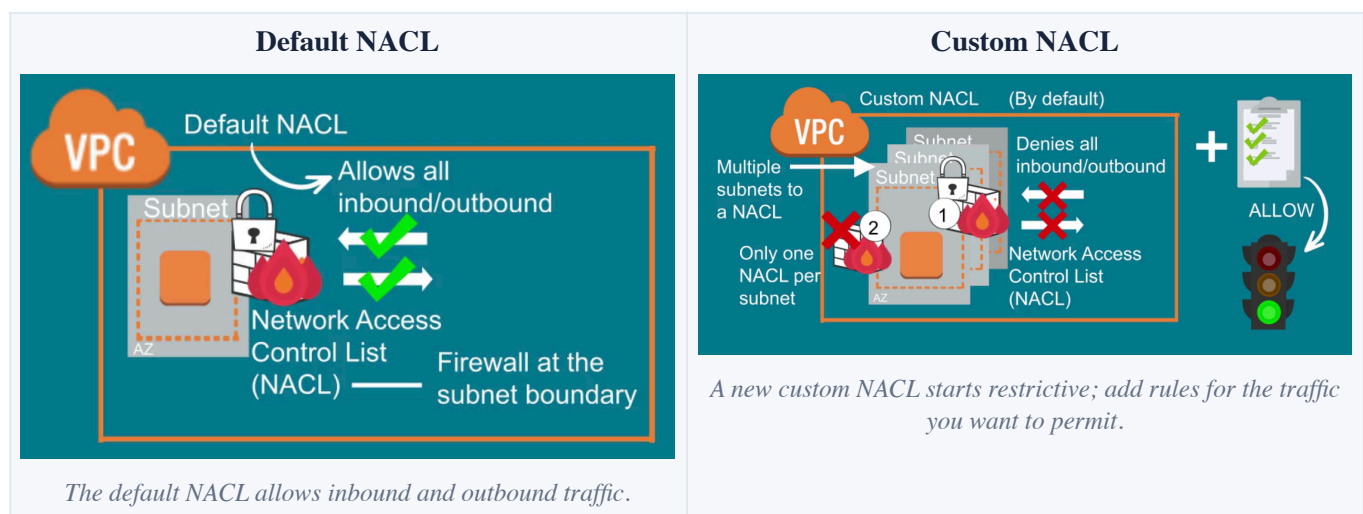
For example, outbound internet traffic may be allowed by the Security Group, but without a route to a NAT or Internet Gateway, it cannot reach the internet.

What if we want to protect the whole subnet?

A Network Access Control List (NACL) controls traffic at the subnet boundary. Every subnet is associated with a NACL. The default NACL that comes with a VPC allows all inbound and outbound traffic, while a new custom NACL denies traffic until matching allow rules are added.

Imagine you wanted to block a specific IP address from reaching anything inside a subnet.

You could manage rules on each individual resource, but what if you wanted one rule at the **subnet boundary** that applies to everything inside?



One NACL can be associated with multiple subnets, but each subnet can be associated with only one NACL at a time. NACLs are stateless, so the necessary inbound and outbound traffic must both be permitted by rules.

How NACL rule priority works

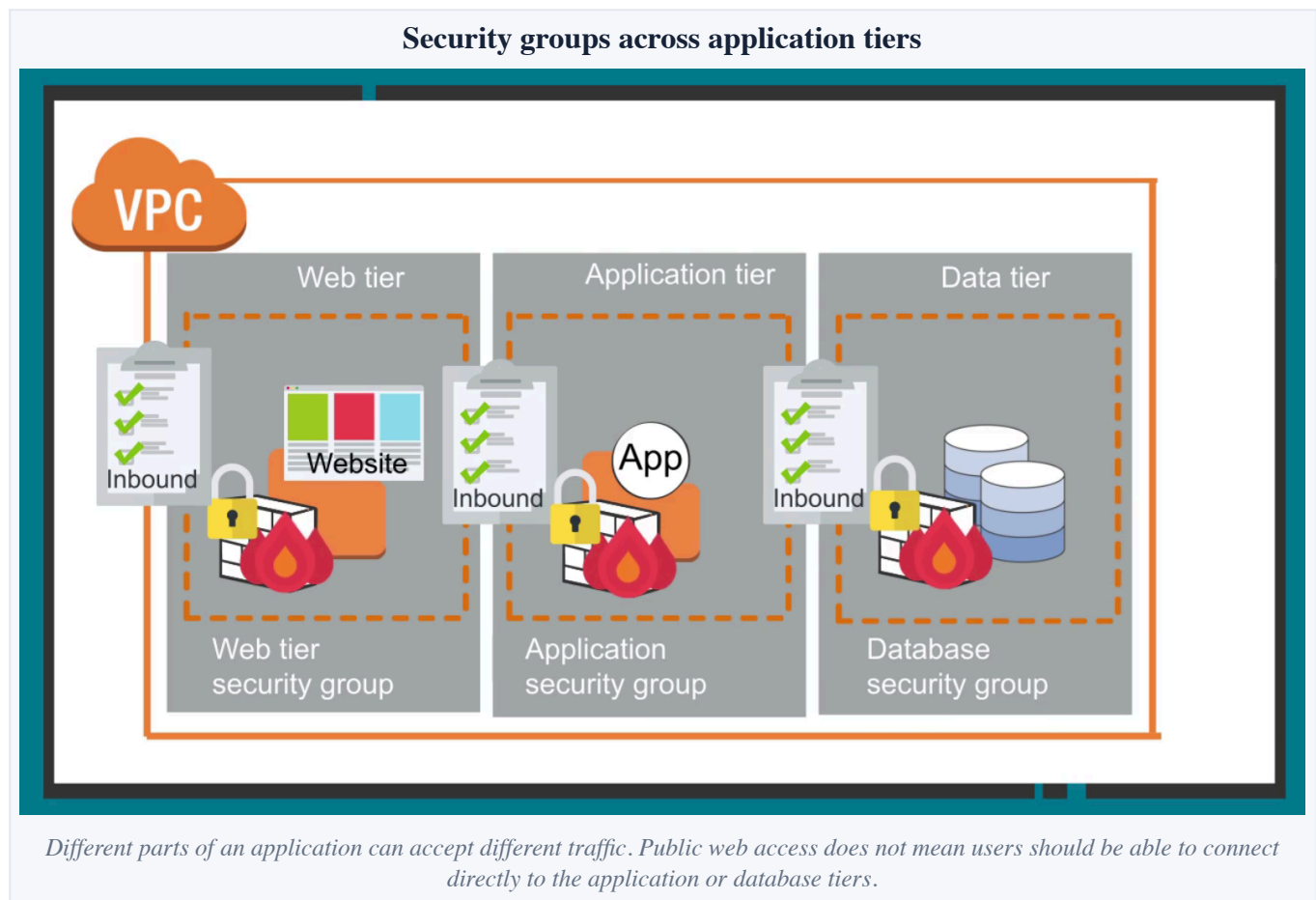
NACL rules are evaluated from the lowest rule number upward. The first matching rule decides whether the traffic is allowed or denied. The number is therefore a priority, not just a label.

Rule #	Type	Protocol	Port	Source	Action
100	HTTP	TCP	80	0.0.0.0/0	ALLOW
110	HTTPS	TCP	443	0.0.0.0/0	ALLOW
120	SSH	TCP	22	203.0.113.0/24	ALLOW
200	All traffic	All	All	0.0.0.0/0	DENY

Security Group vs NACL

	Security Group	NACL
Protects	Resource / network interface	Subnet boundary
Rules	Allow only	Allow and deny
State	Stateful	Stateless
Return traffic	Automatically recognized	Must be permitted by rules
Association	Resources / interfaces	Subnets
Rule priority	No numbered priority	Lowest rule number wins

Stateful means a **Security Group** remembers permitted connections, so response traffic is automatically recognized. **Stateless** means a **NACL** evaluates each direction separately, so the required inbound and outbound traffic must both be permitted.



Imagine allowing customers to reach your website on port 443. That does not mean those customers should also be able to connect directly to the database. Different security boundaries let each part of the application accept only the traffic it actually needs.

What to remember

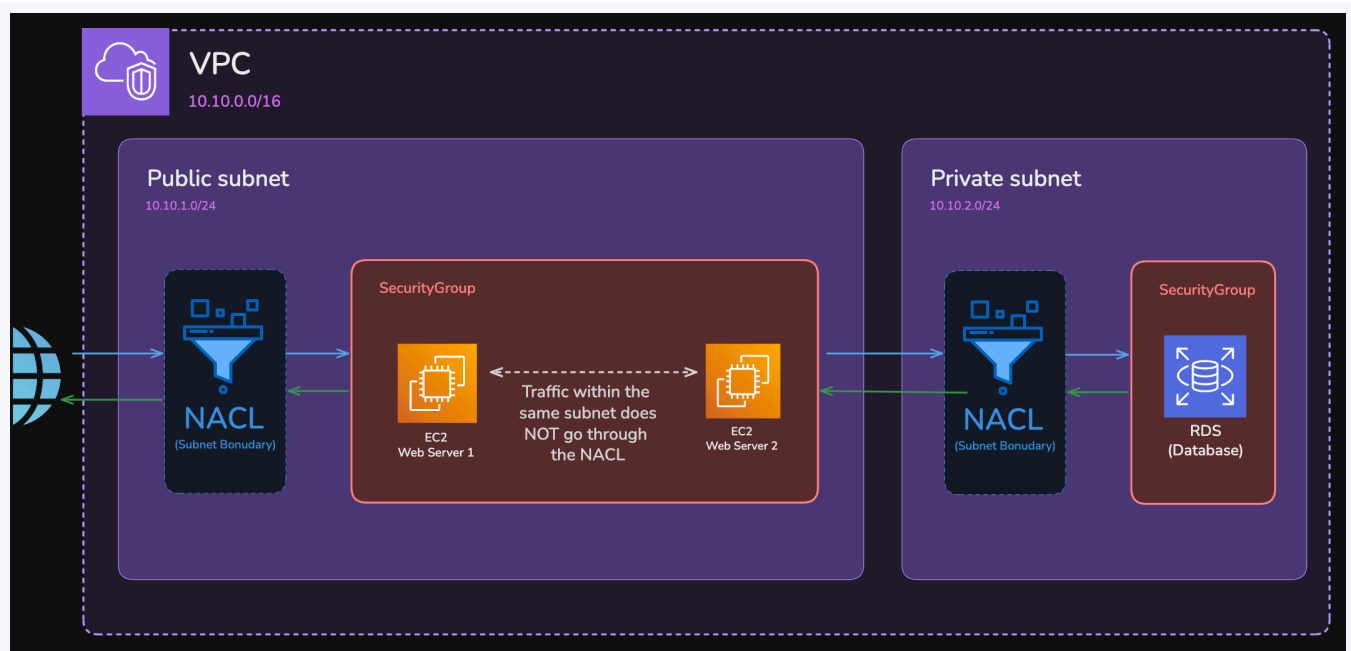
- Routes determine where traffic can go.
- Security Groups control allowed traffic at the resource level.
- NACLs control traffic at the subnet boundary.
- Security Groups are stateful and use allow rules.
- NACLs are stateless, support allow and deny rules, and evaluate numbered rules in priority order.

Putting it all together

Imagine a customer opens our website.

Once traffic reaches the **public subnet**, its NACL can filter traffic crossing the subnet boundary. The web server's **Security Group** then determines whether the connection is allowed to reach the server.

When the web server communicates with the database, traffic crosses into the **private subnet**, where its NACL and the database's own **Security Group** provide separate layers of control.



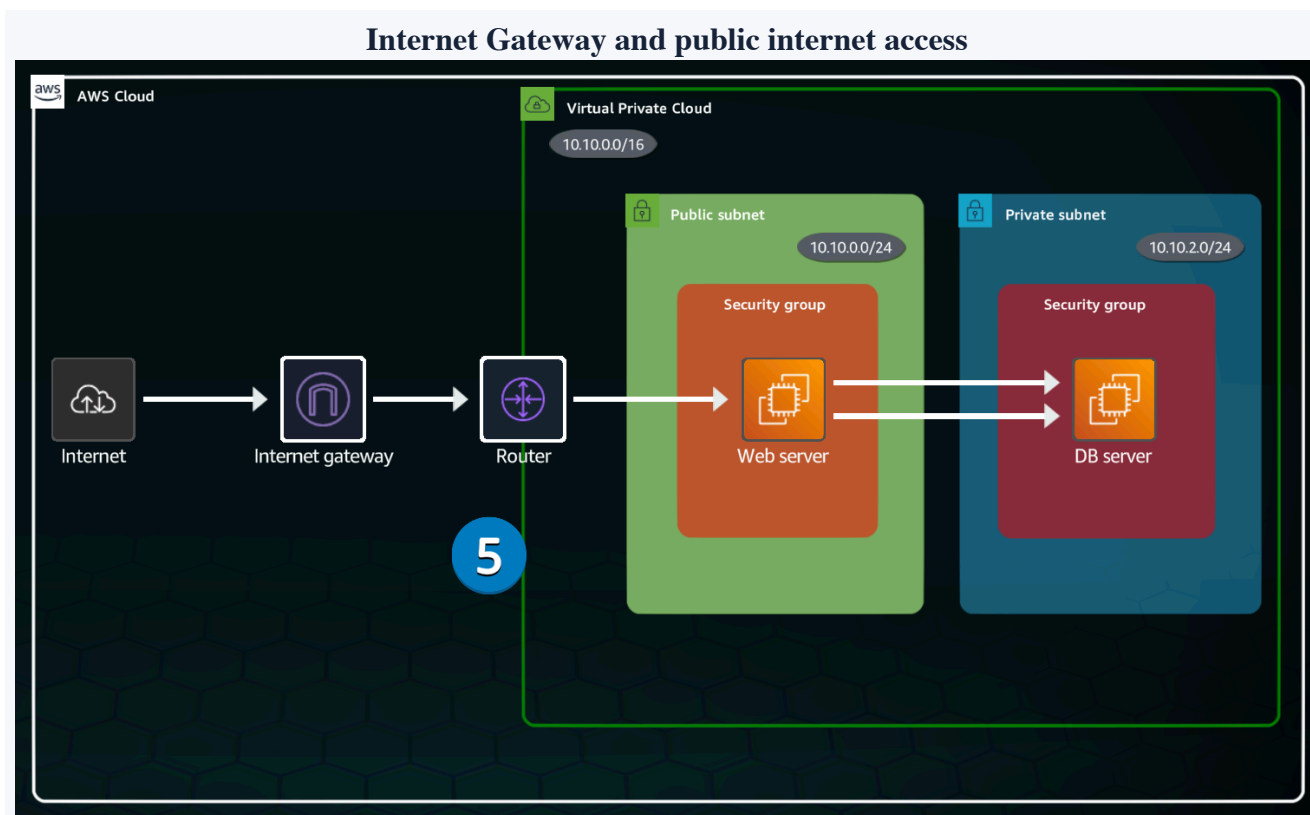
The **NACL protects the subnet boundary**, while the **Security Group protects the resources inside it**.

Notice that both web servers share the same Security Group and are affected by the same subnet NACL. However, traffic between the two web servers stays **within the same subnet**, so it does not cross the NACL boundary.

When the web server communicates with the database, the traffic **leaves the public subnet and enters the private subnet**. This means the relevant NACL rules are evaluated as those subnet boundaries are crossed, while the Security Groups control whether the individual resources accept the traffic.

Internet access: Internet Gateways and NAT

So far, we have decided **where traffic can go** using route tables. But a route alone cannot connect our VPC to the internet.



*The route table sends internet-bound traffic from the public subnet to the Internet Gateway.
The private subnet has no direct route that makes it reachable from the internet.*

The image shows the internet path we want for our web server: customers need to initiate connections from outside the VPC and reach the public subnet.

But our **private resources have a different requirement**. We may not want the internet to initiate connections to them, while still allowing them to reach the Internet when necessary, for example to download updates or call an external API.

These two requirements lead us to two different solutions: an **Internet Gateway** for direct internet connectivity and a **NAT Gateway** for outbound access from private resources.

Internet Gateway: connecting the VPC to the internet

An Internet Gateway (IGW) is attached to a VPC and provides a path between that VPC and the internet. AWS describes it as horizontally scaled, redundant and highly available. It supports both IPv4 and IPv6 traffic.

For an instance in a public subnet to communicate with the internet, the pieces have to line up:

- Attach an Internet Gateway to the VPC.
- Add a route in the subnet route table that points internet-bound traffic to the Internet Gateway.
- Give the instance a public IPv4 address, an Elastic IP address, or IPv6 addressing as appropriate.
- Allow the required traffic through the Security Group and NACL.

An Elastic IP address is a static public IPv4 address. Unlike an automatically assigned public IPv4 address, it can remain associated with your AWS resources until you release it.



PUBLIC INTERNET PATH

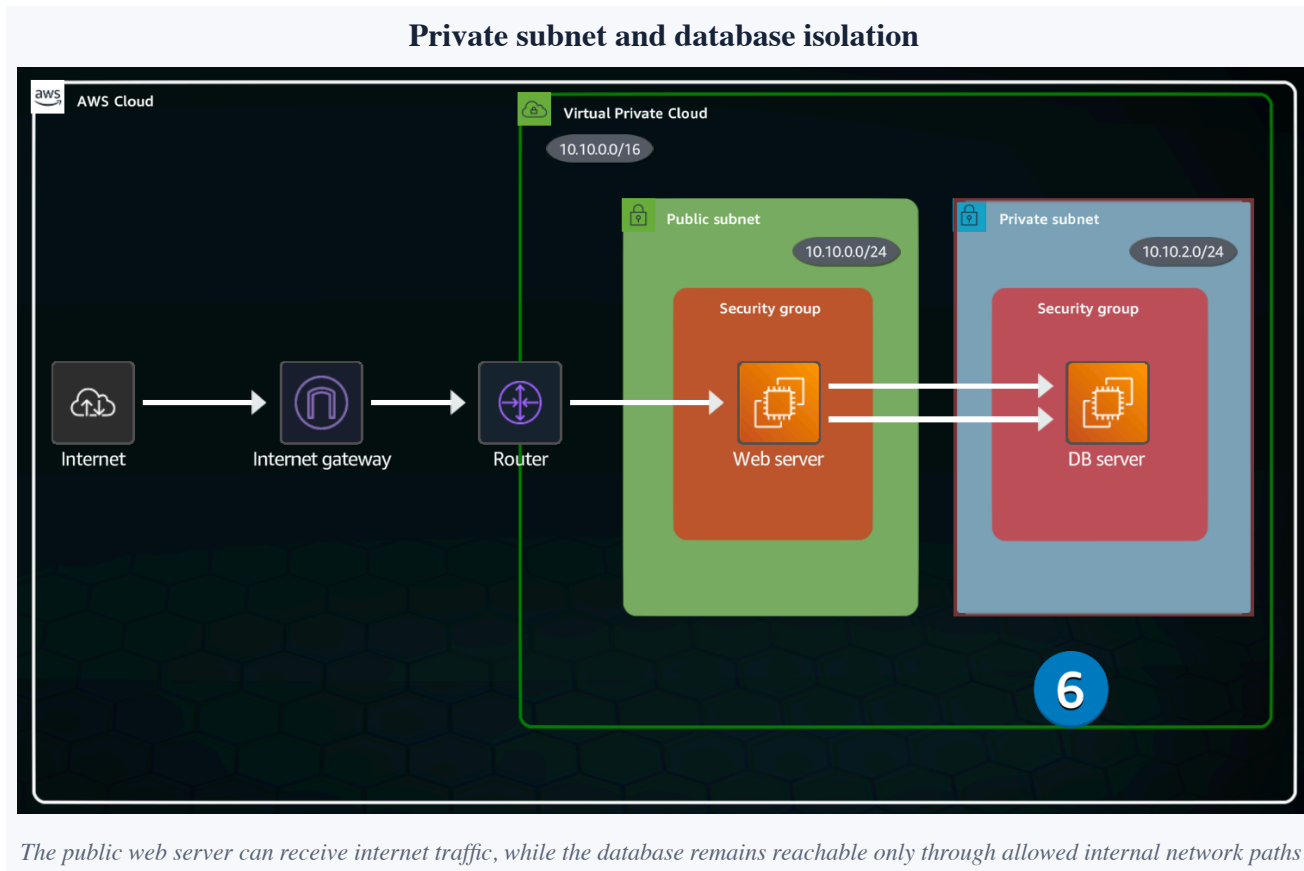
What makes a subnet public?

A subnet is not public just because it contains a web server. Its route table must provide a path to an Internet Gateway, and the resource also needs suitable public addressing and security rules.

Internet Gateways are designed to scale and remain available while providing IPv4/IPv6 connectivity for a VPC.

Private subnet: keep the database off the internet

The database server sits in a private subnet. Its route table does not provide a direct path from the internet, so internet users cannot initiate connections to it.



This separation reduces exposure.

The web server can be internet-facing because customers need to reach it, while the database only needs to accept traffic from the application side of the architecture. **Security Groups** and **NACLs** still decide which traffic is allowed; the private route configuration removes the direct internet path.

If the private database server ever needs outbound internet access, such as for updates, it can use a **NAT Gateway** without becoming directly reachable from the internet.

02 Practice Lab

Build the VPC traffic path

The theory above explains what each networking component does. In the practice lab, we now follow the traffic path in the AWS console and change the configuration ourselves.

In the practice lab we will

- Explore the components that make up a Virtual Private Cloud (VPC).
- Inspect the subnet and the route table associated with it.
- Change the route for internet-bound traffic from a NAT Gateway to an Internet Gateway.
- Configure Security Group inbound rules so the web server and database accept only the traffic they need.

1. Start with the instance and its network

Amazon VPC gives you a logically isolated network inside AWS. A subnet is a smaller IP range inside that VPC and belongs to one Availability Zone. AWS resources such as EC2 instances are placed into these subnets.

Inspect the instance, public address and subnet

Instances (2) Info

Last updated less than a minute ago

Connect

Instance state

Actions

Launch instances

Saved filter sets

Choose filter set

Find Instance by attribute or tag (case-sensitive)

< 1 >

☐	Name	Instance ID	Instance state	Instance type	Status check	Availability Zone	Public IPv4 DNS
☐	Web Server	i-00e3e525e9f68eb5e	Running	t3.small	Initializing	us-east-1a	ec2-34-203-140-12
☐	DB Server	i-0bade82b4fc7d0f1d	Running	t3.small	Initializing	us-east-1a	ec2-13-220-97-189

Details

Status and alarms

Monitoring

Security

Networking

Storage

Tags

VPC ID

[vpc-00798d7006fca1cea \(network-concepts/VPC\)](#)

Availability zone ID

[use1-az4](#)

Subnet ID

[subnet-06000dfc8a08fee78 \(WebServerSubnet\)](#)

Outpost ID

–

Availability zone

[us-east-1a](#)

We have an established VPC in the network tab... And a public IPv4 however..

Public IPv4 address

34.203.140.120 | [open address](#)

Instance state

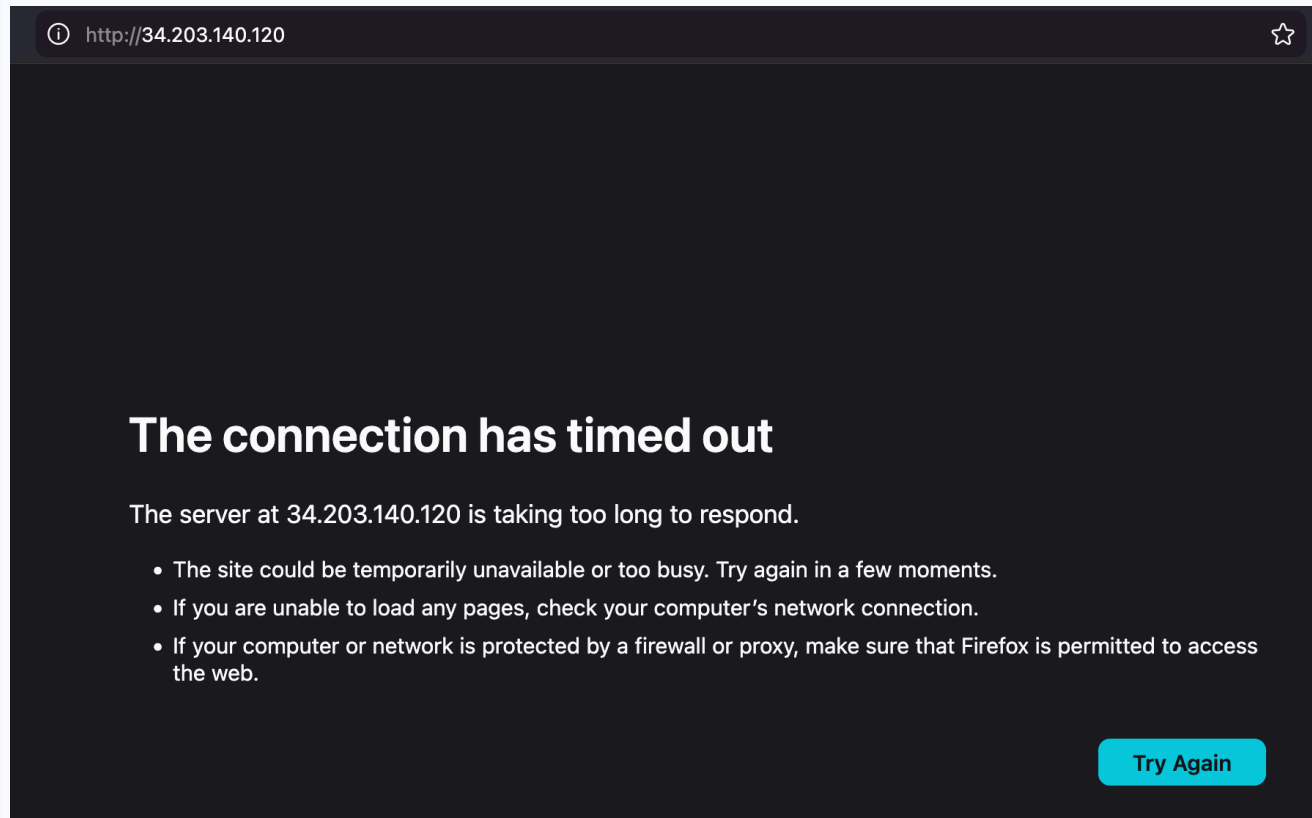
Running

Follow the instance from its overview to its public IPv4 address and the subnet where it is placed.

A public IP does not guarantee reachability

The browser connection still times out even though the instance shows a public IPv4 address. That is an important networking lesson: an address tells us how a resource can be identified, but routing and security still decide whether traffic can actually reach it.

Connection timeout



The failed connection is the clue that another part of the network path is still missing or blocking traffic.

2. Follow the subnet to its route table

Our web server has a public IP address, but users still cannot reach it.

So what path is its network traffic actually using?

The EC2 instance is placed inside a **subnet**. That subnet is associated with a **route table**, which tells AWS where traffic should be sent.

Let's follow the instance to its subnet and inspect those routes.

Subnet and route table

Subnets (1) [Info](#)

Last updated 1 minute ago [Actions](#) [Create subnet](#)

Subnet ID: subnet-06000dfc8a08fee78 [X](#)

[Clear filters](#)

< 1 > [Settings](#)

☐	Name	Subnet ID	State	VPC	Block Public...	IP
☐	WebServerSubnet	subnet-06000dfc8a08fee78	Available	vpc-00798d7006fca1cea netw...	Off	10

The web server is inside **WebServerSubnet**.
Its routing determines where network traffic can go.

Our EC2 instance does not exist on the network by itself.

When it was launched, it was placed inside **WebServerSubnet**, which gives it a place within the VPC's network.
By following the instance's Subnet ID, we can open that subnet and inspect the networking configuration with it.

*If a user on the Internet tries to reach our web server, where does this subnet send that traffic?
To answer that, you need to look at the subnet's route table.*

Subnets (1/1) [Info](#) Last updated 2 minutes ago [Actions](#) [Create subnet](#)

Find subnets by attribute or tag

Subnet ID: subnet-06000dfc8a08fee78 [Clear filters](#)

☒	Name	Subnet ID	State	VPC	Block Public...	IP
☒	WebServerSubnet	subnet-06000dfc8a08fee78	Available	vpc-00798d7006fca1cea netw...	Off	10

subnet-06000dfc8a08fee78 / WebServerSubnet [Settings](#) [Down Arrow](#)

[Details](#) [Flow logs](#) [Route table](#) [Network ACL](#) [CIDR reservations](#) [Sharing](#) [Tags](#) [Related resources](#)

Route table: [rtb-07b552eb902d36a05 / RouteTable2](#) [Edit route table association](#)

Routes (2) [Filter routes](#) [Settings](#)

Destination	Target
10.10.0.0/16	local
0.0.0.0/0	nat-0692196be634de0ad



Every subnet is associated with a **Route Table**.

The route table contains rules that tell AWS where network traffic should be sent based on its destination.

Our WebServerSubnet currently has two routes:

10.10.0.0/16 is local, keeps VPC traffic internal
0.0.0.0/0 is NAT Gateway which provides outbound internet access.

This reveals a problem:

A NAT Gateway provides outbound internet access,
 it doesn't allow users on the internet to initiate a connection to our web server.

We can see that WebServerSubnet is associated with RouteTable2.
 To inspect and modify hits route table directly, open the VPC console > Route tables

Route tables (1) [Info](#) Last updated 5 minutes ago [Actions](#) [Create route table](#)

Find route tables by attribute or tag

Route table ID: rtb-07b552eb902d36a05 [Clear filters](#)

☐	Name	Route table ID	Explicit subnet associ...	Edge associations	Main	VPC
☐	RouteTable2	rtb-07b552eb902d36a05	2 subnets	-	No	vpc-00798d7006

WebServerSubnet uses RouteTable2, which controls where its traffic is directed. Select it.

Route tables (1/1) Info

Last updated 7 minutes ago

Actions

Create route table

Route table ID : rtb-07b552eb902d36a05

Clear filters

< 1 >

⚙

☒	Name	Route table ID	Explicit subnet associ...	Edge associations	Main	VPC
☒	RouteTable2	rtb-07b552eb902d36a05	2 subnets	-	No	vpc-00798d7006

rtb-07b552eb902d36a05 / RouteTable2

⚙

⌵

Details

Routes

Subnet associations

Edge associations

Route propagation

Tags

Related resources

Routes (2)

Both

Edit routes

Destination	Target	Status	Propagated	Route Origin
0.0.0.0/0	nat-0692196be634de0ad	Active	No	Create Route
10.10.0.0/16	local	Active	No	Create Route Table

We have now found the problem.

The route used for destinations outside the VPC, 0.0.0.0/0, currently points to a NAT gateway

10.10.0.0/16 > local

This is the private IP range of our VPC.

If the destination is inside our VPC, keep the traffic inside the VPC.

A user on the internet can NEVER initiate a new connection through the NAT Gateway to reach our web server.

A NAT Gateway is designed to let resources go out to the internet without allowing the internet to initiate connections back to them.

To make the web server publicly reachable, we will change this to the Internet Gateway.

Destination	Target	Meaning
10.10.0.0/16	local	Traffic for addresses inside the VPC stays inside the VPC.
0.0.0.0/0	NAT Gateway	All other IPv4 traffic is sent toward the NAT Gateway.

The destination 0.0.0.0/0 is CIDR notation for all IPv4 addresses. Because the more specific 10.10.0.0/16 route is checked for VPC traffic, 0.0.0.0/0 effectively becomes the fallback route for destinations outside the VPC.

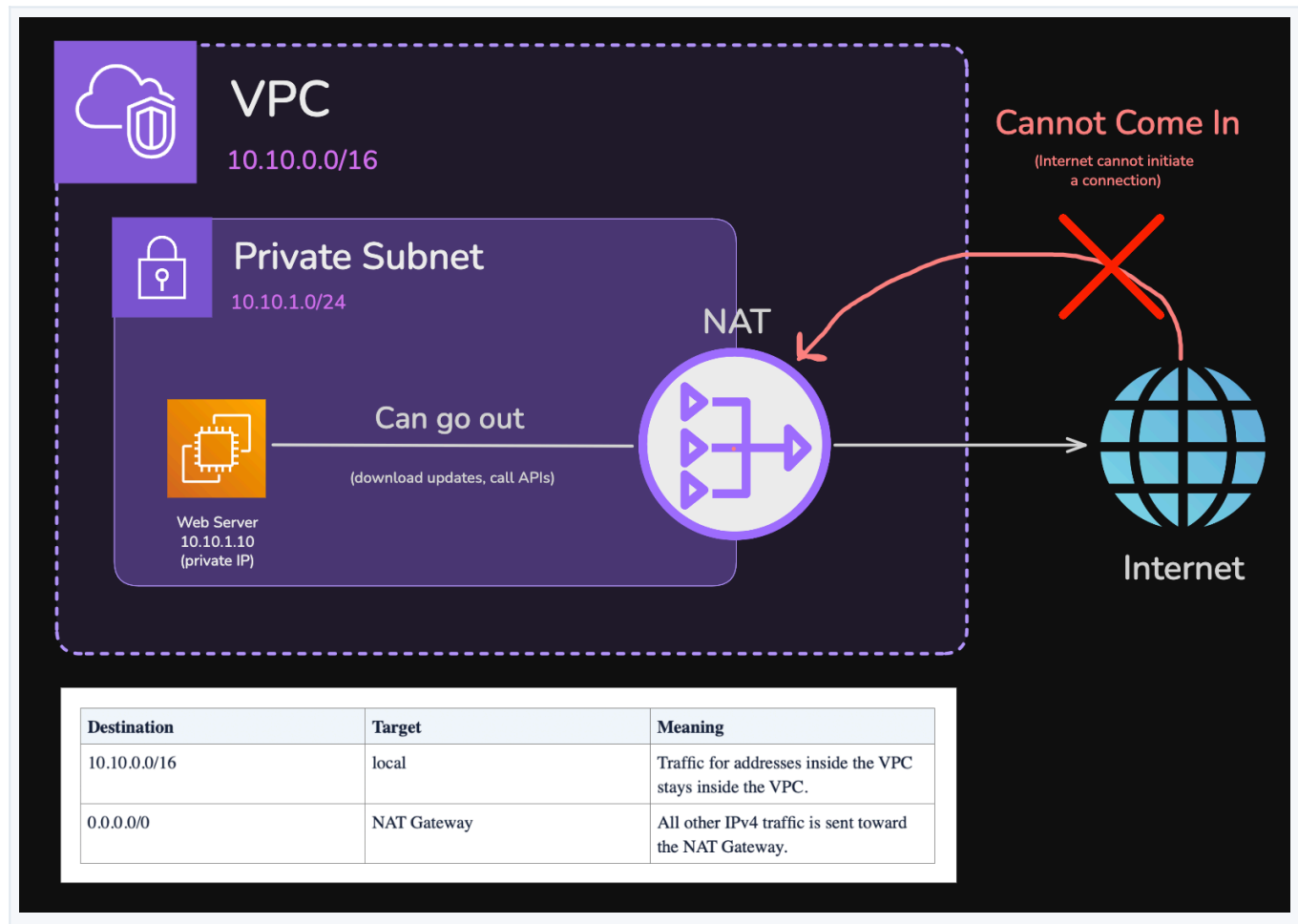
Why the NAT route does not make the server public

A NAT Gateway lets a private EC2 instance start connections to the internet, for example to download updates or call an external API.

But it does not let someone on there internet start a new connection to that EC2 instance.

NAT gives the server outbound internet access, but it does not make the server publicly reachable.

If users need to open the website from their browser, the subnet needs a direct path through an Internet Gateway instead.



A route table works by looking at the destination IP address of traffic and deciding where to send it.

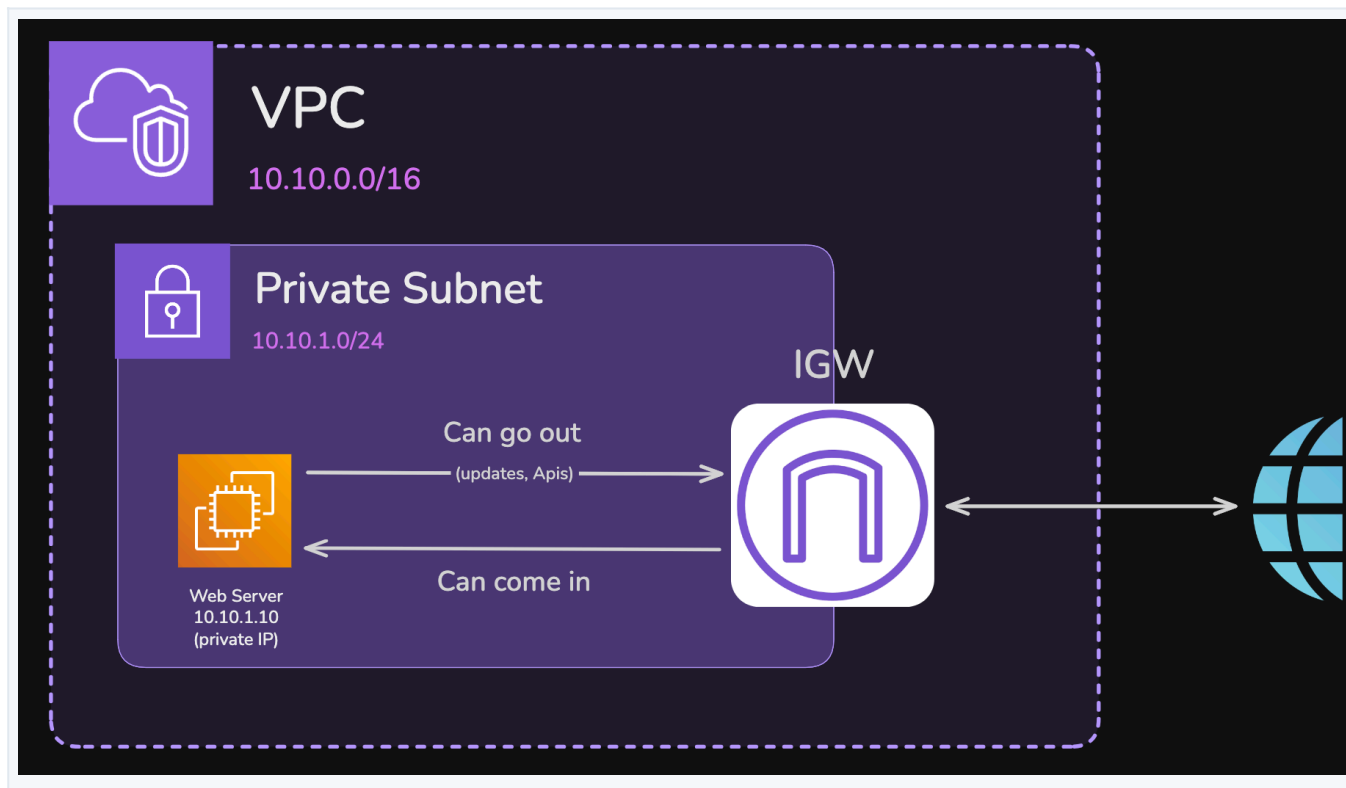
Note that `10.10.0.0/16 > local`

That means: “is the destination an address inside our VPC (10.10.x.x)? Keep it local. Don’t send traffic anywhere else.”

What makes things interesting is that **ALL** other IPv4 addresses that do **NOT** match this rule, are sent to the NAT Gateway.

So if an accountant within this network tried contacting IP 8.8.8.8 (which isn’t inside 10.10.0.0/16 range), it’ll be sent to the NAT Gateway. This ruleset is defined within the **Route Table**.

That means that we could replace our **NAT** with an **Internet Gateway**.



How to think about the route table

The **route table decides which network path the subnet can use**. When our web server communicates with another IP address, AWS checks the route table to determine where the traffic should go.

A **NAT Gateway** lets a private server initiate connections to the internet, such as downloading updates or calling external APIs, without allowing the internet to initiate connections back.

A public web server has a different requirement: **users need to initiate connections to it**. Its subnet therefore needs a route to an **Internet Gateway**, along with the appropriate public addressing and security rules.

If we remove the NAT route without providing another internet route, the server **loses its path to the internet**. But does every server need one? A simple server that only serves files already stored on it may have no reason to initiate internet connections at all.

3. Replace the NAT route with an Internet Gateway

In this lab we remove the NAT Gateway route and point `0.0.0.0/0` to the VPC's Internet Gateway instead. This changes the subnet's internet path.

Change the internet route

Edit routes: rtb-07b552eb902d36a05 / RouteTable2

Destination	Target	Status	Propagated	Route Origin
10.10.0.0/16	local Q local	Active	No	CreateRouteTable

Add route

Cancel

Preview

Save changes

With the removal of `0.0.0.0/0`, the NAT Gateway is no more.
The NAT Gateway lets the server initiate connections out to the internet.
It does not let internet users initiate connections to the web server.

Because our goal is to make the web server publicly reachable, this route does not provide
the type of internet access we need.

Edit routes: rtb-07b552eb902d36a05 / RouteTable2

Destination	Target	Status	Propagated	Route Origin
10.10.0.0/16	local Q local	Active	No	CreateRouteTable
Q 0.0.0.0/0	Internet Gateway Q igw-027b427dfb94e936e	-	No	CreateRoute

Add route

Cancel

Preview

Save changes

Add
0.0.0.0/0 Internet Gateway,
giving the subnet a direct route to and from the internet.

Routes (2)

Both ▼ Edit routes

< 1 > ⚙

Destination	Target	Status	Propagated	Route Origin
0.0.0.0/0	igw-027b427dfb94e936e	✔ Active	No	Create Route
10.10.0.0/16	local	✔ Active	No	Create Route Table

*The Internet Gateway route is now Active.
The web server can be publicly reachable when its public ip and security rules also allow it.*

Destination	Target	Meaning
10.10.0.0/16	local	Keep VPC traffic inside the VPC.
0.0.0.0/0	Internet Gateway	Send unmatched IPv4 traffic toward the internet.

You can mentally read this as: inside our VPC, keep it local; anything else, send it toward the Internet Gateway. The Internet Gateway gives the VPC a direct internet path, but the instance still needs suitable addressing and security rules. For IPv4, that normally means a public IPv4 address or Elastic IP. An Elastic IP is a static public IPv4 address that you control in your AWS account.

Pricing

There is no hourly charge for the Internet Gateway itself, although normal AWS data-transfer charges can still apply. NAT Gateways, by contrast, have their own usage charges while provisioned.

4. Open the Security Group

Routing gets traffic to the right network path.
The Security Group is the next checkpoint: it decides which traffic is allowed to reach the instance.

Navigate to the instance Security Group

Instances (1/2) [Info](#)

Last updated 32 minutes ago

[Connect](#)

[Instance state](#)

[Actions](#)

[Launch instances](#)

Saved filter sets

Choose filter set

1

	Name	Instance ID	Instance state	Instance type	Status check	Availability Zone	Public IPv4 DNS
☒	Web Server	i-00e3e525e9f68eb5e	Running	t3.small	Initializing	us-east-1a	ec2-34-203-140-12
☐	DB Server	i-0bade82b4fc7d0f1d	Running	t3.small	Initializing	us-east-1a	ec2-13-220-97-189

i-00e3e525e9f68eb5e (Web Server)

Details

Status and alarms

Monitoring

Security

Networking

Storage

Tags

▼ Security details

IAM role
 [ec2_app_role-19a4e420](#)

Owner ID
 408687975267

Launch time
Thu Aug 27 2026 13:43:42 GMT+0200 (Central European Summer Time)

Security groups
 [sg-0c8f85c5a4a66a9d2 \(WebServerSecurityGroup\)](#)

The web server uses WebServerSecurityGroup.

Security Groups control which traffic can enter or leave the instance.

sg-0c8f85c5a4a66a9d2 - WebServerSecurityGroup

Actions

Details

Security group name

WebServerSecurityGroup

Security group ID

sg-0c8f85c5a4a66a9d2

Description

Security Group for Web Server

VPC ID

vpc-00798d7006fca1cea

Owner

408687975267

Inbound rules count

0 Permission entries

Outbound rules count

3 Permission entries

Inbound rules

Outbound rules

Sharing

VPC associations

Related resources - new

Tags

Inbound rules

Manage tags

Edit inbound rules

Search

< 1 >

Name

Security group rule ID

IP version

Type

Protocol

Port range

No security group rules found

The Security Group currently has **no inbound rules**, so incoming connections are blocked.

Next up is allowing the web traffic to hit the server.

Allow HTTP to the public web server

We add an inbound rule for HTTP on TCP port 80 with source Anywhere-IPv4. This explicitly allows any IPv4 address to attempt an HTTP connection to the web server.

Add the HTTP inbound rule

Edit inbound rules

Inbound rules control the incoming traffic that's allowed to reach the instance.

Inbound rules

Security group rule ID

Type

Protocol

Port range

Source

Description - optional

-

HTTP

TCP

80

Anyw...

Add rule

0.0.0.0/0

Delete

Rules with source of 0.0.0.0/0 or ::/0 allow all IP addresses to access your instance. We recommend setting security group rules to allow access from known IP addresses only.

Cancel

Preview changes

Save rules

AWS warns about broad sources because 0.0.0.0/0 exposes that port to the public internet.

For a public web server, allowing HTTP or HTTPS from the internet can be expected. The same broad source would usually be inappropriate for sensitive management ports or a database.

5. Allow only the web tier to reach MySQL

The database should not accept MySQL connections from the whole internet. Instead, its Security Group can allow TCP port 3306 with the web server's Security Group as the source.

Restrict database access to the web Security Group

Inbound rules

Outbound rules

Sharing

VPC associations

Related resources - new

Tags

Outbound rules (3)

Manage tags

Edit outbound rules

Search

	Name	Security group rule ID	IP version	Type	Protocol	Port range
☐	-	sgr-0ef0695a58164f1f4	IPv4	HTTPS	TCP	443
☐	-	sgr-092544613b7dae35f	-	MYSQL/Aurora	TCP	3306
☐	-	sgr-02d8ecea36ecd95	IPv4	HTTP	TCP	80

Outbound rules control where the web server is allowed to send traffic.

Edit outbound rules

Outbound rules control the outgoing traffic that's allowed to leave the instance.

Outbound rules

Security group rule ID	Type	Protocol	Port range	Destination	Description - optional
sgr-0ef0695a58164f1f4	HTTPS	TCP	443	Custom	Allow HTTPS traffic for S3, S5
sgr-092544613b7dae35f	MYSQL/Aurora	TCP	3306	Custom	Allow Web Server to connect
sgr-02d8ecea36ecd95	HTTP	TCP	80	Custom	Allow outbound HTTP traffic

Add rule

Rules with destination of 0.0.0.0/0 or ::/0 allow your instances to send traffic to any IPv4 or IPv6 address. We recommend setting security group rules to be more restrictive and to only allow traffic to specific known IP addresses.

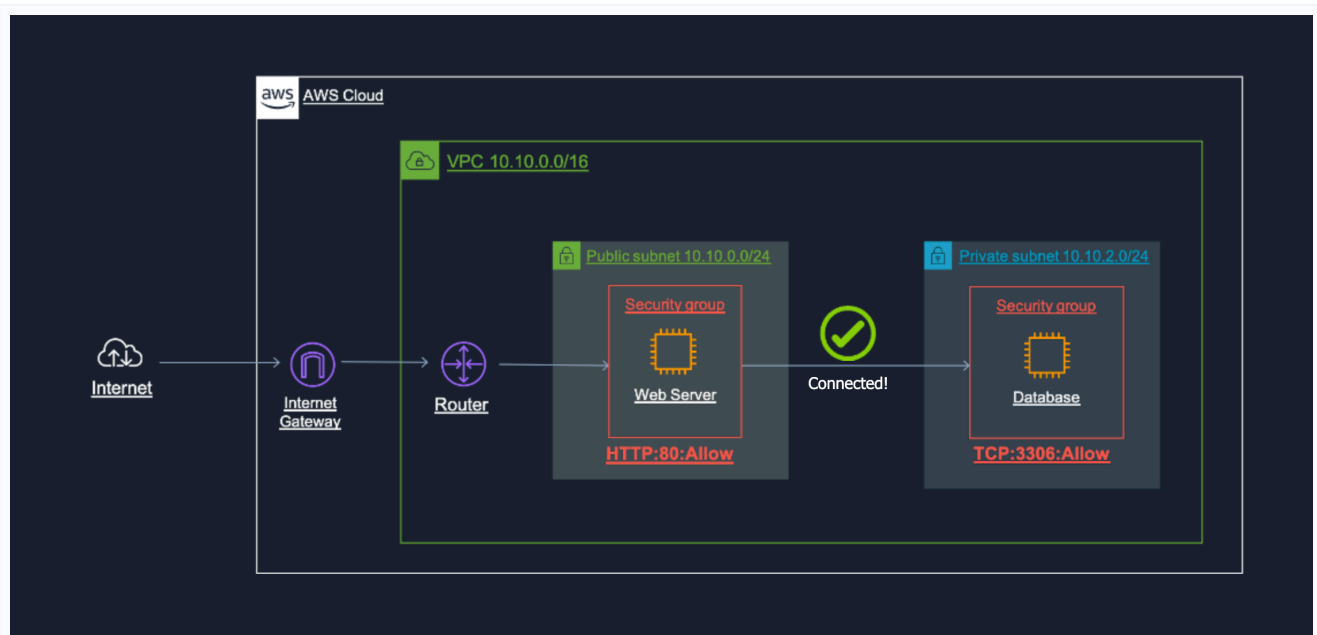
Cancel

Preview changes

Save rules

Keep only the connections the web server needs, including MySQL on TCP 3306.

AWS Cloud • Part II



*Port 3306 is allowed specifically to the database Security Group.
The web server can now communicate with the database without exposing the DB to everyone.*

The lab's final mental model

A VPC is our private AWS network, divided into subnets with defined IP ranges, while route tables direct traffic through gateways such as an Internet Gateway or NAT Gateway, and Security Groups and NACLs act as firewalls controlling what traffic is allowed.